

Control Flow

Rahul Guha || SME || Java FS Training || HCL

Control Statements in Java

Introduction

- Control statements in Java are the instructions that controls or manages the flow of execution of a program based on specific conditions or loops.
- Control Statements are used to:
- Make decisions: Control program flow based on conditions (e.g., if, switch).
- Loop through blocks of code: Repeat code execution multiple times (e.g., for, while).
- Jump to a different part of the code: Change the natural flow of execution (e.g., break, continue).

Types of Control Statements:

- Control Statements in Java are divided in 3 main categories:

1. **Decision-Making Statements or Conditional Statements**

- These statements allow the program to make decisions and execute a block of code based on a condition.
- Examples are: if, if-else, if-else if ladder, switch.
- [Click Here to read Conditional Statements more deeply.](#)

Types of Control Statements:

- Control Statements in Java are divided in 3 main categories:

2. Iteration or Looping Statements

These statements allow the execution of a block of code multiple times until a condition is satisfied.

Examples are: for, while, do-while.

[Click Here to read Looping Statements more deeply.](#)

Types of Control Statements:

- Control Statements in Java are divided in 3 main categories:

3. **Jump Statements**

These statements are used to alter the flow of control by jumping to a specific part of the program.

Examples are: break, continue, return.

[Click Here to read Jump Statements more deeply.](#)

Decision-Making Statements or Conditional Statements

"Is it raining ?"

"No" - Then lets play cricket. || "Yes" - Ohh, we cant play cricket.

Just like in real life, in programming also, we face situations where we need to make decisions based on certain conditions. In Java, we use conditional statements to decide which part of the code should run depending on the condition.

Conditional Statements helps us to:

1. Control the flow of the program. 2. Decide which block of code should be executed when certain conditions are met.

Examples of Conditional Statements in Java:

1. if: Runs code if a condition is true.
2. if-else: Runs one block of code if true, and another block if false.
3. if-else if ladder: Checks multiple conditions one by one and runs the block of code for the first true condition.
4. switch: Chooses a block of code to run based on specific cases.

Iteration Statements

Looping Statements are also known as "Iteration Statements".

Looping Statements allow us to repeat a block of code multiple times, making our programs more efficient and reducing redundancy.

Loops are essential for handling repetitive tasks like:

1. printing data multiple times
2. executing tasks until a specific condition is met
3. iterating through arrays
4. and many more....

Examples of Looping Statements in Java:

1. **for:** Repeats a block of code a specific number of times.
2. **while:** Executes a block of code as long as a specified condition is true.
3. **do-while:** Executes a block of code once, then repeats it as long as the condition is true.
4. **for-each (Enhanced For Loop):** Iterates over elements in an array or collection.

Jump statements

Jump statements transfer the program's control from one part of the code to another, skipping the lines in between.

By default, Java programs execute statements line by line from top to bottom, but jump statements allows us to:

- Break the usual execution order (e.g., exit a loop early using break).
- Skip parts of the code (e.g., skip an iteration using continue).
- Return control to the method caller (e.g., exit a method and optionally return a value using return).

Examples of Jump Statements in Java:

- break: Stops the loop completely when a condition is true.
- continue: Skips the current loop step and moves to the next one
- return: Ends the method and sends a result back, if needed.

Return statements

"return" Statements in Java

The return statement is used to exit from a method and optionally send a value back to the method's caller.

It is essential for returning a result from a method or terminating the execution of a method before it reaches its end.

The usage of the return keyword in Java, categorized into two main cases:

- Methods returning a value: The return keyword sends a value back to the caller.
- Methods not returning a value (Void methods)
- Without return: return stops the method early.
- With void: return ends the method without returning a value.

Complete Comparison Table

Control Statement	Category	Main Purpose	Condition Required?	Executes Minimum	Best Used When
if	Decision	Execute code when a condition is true	Yes	0 times	One condition
if-else	Decision	Choose between two blocks	Yes	1 block	Two alternatives
else-if	Decision	Check multiple conditions	Yes	0 times	Multiple related conditions
switch	Decision	Select one block from multiple choices	Value/expression	0 or 1 case	Menu/options/fixed values
for	Loop	Repeat code a known number of times	Yes	0 times	Counter-controlled loops
while	Loop	Repeat while condition is true	Yes	0 times	Condition-controlled loops
do-while	Loop	Execute first, then check condition	Yes	1 time	At least one execution required
break	Jump	Immediately terminate loop/switch	No	—	Stop execution
continue	Jump	Skip current iteration	No	—	Skip selected iterations
return	Jump	Exit method and optionally return value	No	—	Return from a method

Decision-Making Statements

Feature	if	if-else	else-if	switch
Conditions	One	One	Multiple	Multiple cases
Alternative	No	Yes	Yes	Yes
Condition type	Boolean expression	Boolean expression	Boolean expressions	Expression/value matching
Best for	Simple decision	Two choices	Multiple conditions	Fixed choices
Readability	High	High	Good for related conditions	High for menu-like choices
Fall-through	No	No	No	Possible without break in traditional switch
Example	Age check	Pass/Fail	Grade calculation	Menu selection